

实验一、VMWare 虚拟机操作系统安装与应用

本次实验的两个虚拟机我选择的是 kali 和 ubuntu22.04

1. VMWare 虚拟机安装操作系统
2. 创建两台虚拟机并配置
3. 实现网络互通

以下是关于 1, 2, 3 问题的回答

```
[ygzl@dnherre:~] - (~/.CTF)
└─ ifconfig
br-bc2958d2ffb1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    inet6 fe80::42:ddff:fe71:63d3 prefixlen 64 scopeid 0x20<link>
    ether 02:42:dd:71:63:d3 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:47:3c:ec:f1 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ens32: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.99 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::20c:29ff:fe29:51ee prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:29:51:ee txqueuelen 1000 (Ethernet)
    RX packets 340 bytes 29728 (29.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 346 bytes 32705 (32.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
└─ ping baidu.com
PING baidu.com (39.156.70.37) 56(84) bytes of data.
64 bytes from 39.156.70.37 (39.156.70.37): icmp_seq=1 ttl=128 time=17.3 ms
64 bytes from 39.156.70.37 (39.156.70.37): icmp_seq=2 ttl=128 time=15.8 ms
64 bytes from 39.156.70.37 (39.156.70.37): icmp_seq=3 ttl=128 time=16.1 ms
64 bytes from 39.156.70.37 (39.156.70.37): icmp_seq=4 ttl=128 time=16.1 ms
^C
--- baidu.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 15.841/16.332/17.273/0.553 ms
```

Ubuntu 主机 ip 为 192.168.1.99, 可以 ping 通外网

```

(ygkldnherr@kali)-[~]
$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:50:4a:d4:41 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.53 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::20c:29ff:fe9d:3966 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:9d:39:66 txqueuelen 1000 (Ethernet)
    RX packets 295 bytes 18706 (18.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 117 bytes 9682 (9.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 56 bytes 4944 (4.8 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 56 bytes 4944 (4.8 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

```

```

$ ping baidu.com
PING baidu.com (39.156.70.37) 56(84) bytes of data.
64 bytes from 39.156.70.37: icmp_seq=1 ttl=128 time=16.8 ms
64 bytes from 39.156.70.37: icmp_seq=2 ttl=128 time=16.3 ms
64 bytes from 39.156.70.37: icmp_seq=3 ttl=128 time=16.8 ms
64 bytes from 39.156.70.37: icmp_seq=4 ttl=128 time=20.0 ms
64 bytes from 39.156.70.37: icmp_seq=5 ttl=128 time=16.2 ms
64 bytes from 39.156.70.37: icmp_seq=6 ttl=128 time=16.3 ms
64 bytes from 39.156.70.37: icmp_seq=7 ttl=128 time=15.9 ms
64 bytes from 39.156.70.37: icmp_seq=8 ttl=128 time=16.8 ms
64 bytes from 39.156.70.37: icmp_seq=9 ttl=128 time=15.9 ms
64 bytes from 39.156.70.37: icmp_seq=10 ttl=128 time=15.9 ms
^C
— baidu.com ping statistics —
10 packets transmitted, 10 received, 0% packet loss, time 11118ms
rtt min/avg/max/mdev = 15.908/16.696/20.006/1.157 ms

```

Kali 主机的 ip 地址为 192.168.1.53，可以 ping 通外网

```

$ ping 192.168.1.99
PING 192.168.1.99 (192.168.1.99) 56(84) bytes of data.
64 bytes from 192.168.1.99: icmp_seq=1 ttl=64 time=0.615 ms
64 bytes from 192.168.1.99: icmp_seq=2 ttl=64 time=0.408 ms
64 bytes from 192.168.1.99: icmp_seq=3 ttl=64 time=0.534 ms
^C
— 192.168.1.99 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2047ms
rtt min/avg/max/mdev = 0.408/0.519/0.615/0.085 ms

```

可以看到 ping 通 ubuntu 主机

4. 学习使用 iptables 和 firewalld

这两个工具都是用于管理防火墙的工具，但抽象层次不同

首先需要了解 Netfilter 的概念，Linux 内核中真正实现防火墙功能的是 Netfilter 框架。它是在内核级别提供的 hooks，允许对网络数据包进行过滤、修改等操作。

iptables 是一个命令行工具，它提供了一种用户空间的方式来与内核的 Netfilter 框架进行交互、配置和管理规则。可以把它看作是与 Netfilter 对话的直接“语言”。但是它的规则是“静态”的。添加、删除或修改规则意味着直接替换整个规则集，这在复杂的动态环境中可能造成短暂中断。管理和维护大量规则比较繁琐。

```

$ sudo iptables -L -v -n
[sudo] password for ygkldnherr:
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source                   destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source                   destination
0      0 DOCKER-USER  all  --  *      *       0.0.0.0/0               0.0.0.0/0
0      0 DOCKER-ISOLATION-STAGE-1  all  --  *      *       0.0.0.0/0               0.0.0.0/0
0      0 ACCEPT      all  --  *      docker0 0.0.0.0/0               0.0.0.0/0      ctstate RELATED,ESTABLISHED
0      0 DOCKER      all  --  *      docker0 0.0.0.0/0               0.0.0.0/0
0      0 ACCEPT      all  --  docker0 !docker0 0.0.0.0/0               0.0.0.0/0
0      0 ACCEPT      all  --  docker0 docker0  0.0.0.0/0               0.0.0.0/0

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source                   destination

Chain DOCKER (1 references)
pkts bytes target      prot opt in     out     source                   destination

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
pkts bytes target      prot opt in     out     source                   destination
0      0 DOCKER-ISOLATION-STAGE-2  all  --  docker0 !docker0 0.0.0.0/0               0.0.0.0/0
0      0 RETURN      all  --  *      *       0.0.0.0/0               0.0.0.0/0

Chain DOCKER-ISOLATION-STAGE-2 (1 references)
pkts bytes target      prot opt in     out     source                   destination
0      0 DROP        all  --  *      docker0 0.0.0.0/0               0.0.0.0/0
0      0 RETURN      all  --  *      *       0.0.0.0/0               0.0.0.0/0

Chain DOCKER-USER (1 references)
pkts bytes target      prot opt in     out     source                   destination
0      0 RETURN      all  --  *      *       0.0.0.0/0               0.0.0.0/0

```

```

$ sudo iptables -t nat -L -v -n
Chain PREROUTING (policy ACCEPT 7 packets, 1150 bytes)
pkts bytes target      prot opt in     out     source                   destination
0      0 DOCKER      all  --  *      *       0.0.0.0/0               0.0.0.0/0      ADDRTYPE match dst-type LOCAL

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out     source                   destination

Chain OUTPUT (policy ACCEPT 92 packets, 7369 bytes)
pkts bytes target      prot opt in     out     source                   destination
0      0 DOCKER      all  --  *      *       0.0.0.0/0               !127.0.0.0/8    ADDRTYPE match dst-type LOCAL

Chain POSTROUTING (policy ACCEPT 92 packets, 7369 bytes)
pkts bytes target      prot opt in     out     source                   destination
0      0 MASQUERADE  all  --  *      !docker0 172.17.0.0/16           0.0.0.0/0

Chain DOCKER (2 references)
pkts bytes target      prot opt in     out     source                   destination
0      0 RETURN      all  --  docker0 *       0.0.0.0/0               0.0.0.0/0

```

firewalld 是建立在 iptables 之上的一个动态防火墙管理器。它提供了一个更高级、更用户友好的抽象层。它的核心创新是引入了“zones”（区域）和“services”（服务）的概念。

```
└─$ sudo systemctl status firewalld
● firewalld.service - firewalld - dynamic firewall daemon
   Loaded: loaded (/usr/lib/systemd/system/firewalld.service; disabled; preset: disabled)
   Active: active (running) since Thu 2025-09-11 21:07:14 CST; 2s ago
 Invocation: e59eb180faec443abd36fec4d3e1e5ca
    Docs: man:firewalld(1)
   Process: 4305 ExecStartPost=/usr/bin/firewall-cmd --state (code=exited, status=0/SUCCESS)
  Main PID: 4303 (firewalld)
     Tasks: 2 (limit: 9333)
    Memory: 29.4M (peak: 55.6M)
       CPU: 346ms
    CGroup: /system.slice/firewalld.service
            └─4303 /usr/bin/python3 /usr/sbin/firewalld --nofork --nopid

Sep 11 21:07:13 kali systemd[1]: Starting firewalld.service - firewalld - dynamic firewall daemon...
Sep 11 21:07:14 kali systemd[1]: Started firewalld.service - firewalld - dynamic firewall daemon.
```

```
(ygkldnherr@kali)-[~]
└─$ sudo firewall-cmd --get-default-zone
public

(ygkldnherr@kali)-[~]
└─$ sudo firewall-cmd --get-active-zones
public (default)
   interfaces: eth0

(ygkldnherr@kali)-[~]
└─$ sudo firewall-cmd --list-all
public (default, active)
   target: default
 ingress-priority: 0
 egress-priority: 0
 icmp-block-inversion: no
   interfaces: eth0
    sources:
  services: dhcpv6-client ssh
    ports:
  protocols:
 forward: yes
masquerade: no
forward-ports:
source-ports:
 icmp-blocks:
  rich rules:
```

```
(ygkldnherr@kali)-[~]  
$ sudo firewall-cmd --permanent --add-service=ssh  
Warning: ALREADY_ENABLED: ssh  
success  
  
(ygkldnherr@kali)-[~]  
$ sudo firewall-cmd --add-service=http  
success  
  
(ygkldnherr@kali)-[~]  
$ sudo firewall-cmd --add-service=https  
success
```

5. 了解 Bridge 和路由

Bridge 即网桥，工作在数据链路层（第二层），主要是用来连接两个局域网或两个二层网段，把它们当作一个整体进行转发，起到“透明转发”的作用。它不修改 IP 地址，基于 MAC 地址学习转发表。形象的比喻，就像一个“楼层内的门”：不同房间（局域网段）的人通过门能自由走动，但还是在同一层楼里，大家喊话（广播）别人都能听到。

Router 即路由器，工作在 网络层（第三层），主要负责在不同网络之间转发数据，根据 IP 地址和路由表来决定路径，同时会进行分隔广播域。形象的比喻，就像“楼层间的电梯”：不同楼层（网络）之间必须通过电梯才能沟通，而且电梯会检查你要去的楼层号码（IP），不会随便乱送。

6. 在 Linux 环境下编程 C 语言程序实现输出斐波那契数列前 10 项

```
(ygkldnherr@kali)-[~/Documents]
$ cat ./fibonacci.c
#include <stdio.h>

int main(){
    int num1 = 0, num2 = 1, nextNum;
    for(int i = 0; i < 10; i++){
        printf("%d, ", num1);
        nextNum = num1 + num2;
        num1 = num2;
        num2 = nextNum;
    }
    return 0;
}

(ygkldnherr@kali)-[~/Documents]
$ gcc -o fibonacci.out ./fibonacci.c

(ygkldnherr@kali)-[~/Documents]
$ ls
fibonacci.c  fibonacci.out

(ygkldnherr@kali)-[~/Documents]
$ ./fibonacci.out
0, 1, 1, 2, 3, 5, 8, 13, 21, 34,
```